CORDIC Algorithm: Theory and Implementation

Antigravity AI Assistant

March 8, 2026

1 Introduction

CORDIC (COordinate Rotation Digital Computer) is an algorithm to calculate trigonometric, hyperbolic, and other mathematical functions using only shifts and additions.

1.1 Main Uses

The primary applications of the CORDIC algorithm include:

- Realization of rotations.
- Calculation of trigonometric functions $(\sin, \cos)$ and inverse trigonometric functions $(\tan^{-1}(a/b))$.
- Calculation of magnitude $(\sqrt{a^2+b^2})$.
- Extension to hyperbolic functions, natural logarithms, and exponentials.
- Division and multiplication.
- Pervasive use in DSP, Image Processing, 3D Graphics, and Robotics.

2 The Core Idea: Successive Rotations

A perfect rotation of a vector (x_{in}, y_{in}) by an angle θ results in (x_R, y_R) given by:

$$x_R = M_{in}\cos(\beta+\theta) = x_{in}\cos\theta - y_{in}\sin\theta \tag{1}$$

$$y_R = M_{in}\sin(\beta+\theta) = x_{in}\sin\theta + y_{in}\cos\theta \tag{2}$$

where M_{in} is the modulus and β is the initial angle. In matrix form:

$$\begin{bmatrix} x_R \\ y_R \end{bmatrix} = \begin{bmatrix} \cos\theta & -\sin\theta \\ \sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x_{in} \\ y_{in} \end{bmatrix} = \mathrm{ROT}(\theta) \begin{bmatrix} x_{in} \\ y_{in} \end{bmatrix} \tag{3}$$

2.1 Micro-Rotations and Simplifications

The key idea is to decompose the angle θ into a sequence of known elementary micro-rotation angles α_j:

$$\theta = \sum_{j=0}^{\infty} \alpha_j \implies \mathrm{ROT}(\theta) = \prod_{j=0}^{\infty} \mathrm{ROT}(\alpha_j) \tag{4}$$

To avoid multiplications, the rotation is decomposed into a *scaling operation* and a *rotation-extension*:

$$x_R[j+1] = \cos(\alpha_j)(x_R[j] - y_R[j]\tan(\alpha_j)) \tag{5}$$

$$y_R[j+1] = \cos(\alpha_j)(y_R[j] + x_R[j]\tan(\alpha_j)) \tag{6}$$

By choosing elementary angles $\alpha_j = \sigma_j \tan^{-1}(2^{-j})$ with $\sigma_j \in \{-1, 1\}$, the rotation-extension recurrence relies only on additions and shifts:

$$x[j+1] = x[j] - \sigma_j 2^{-j} y[j] \tag{7}$$

$$y[j+1] = y[j] + \sigma_j 2^{-j} x[j] \tag{8}$$

$$z[j+1] = z[j] - \sigma_j \tan^{-1}(2^{-j}) \tag{9}$$

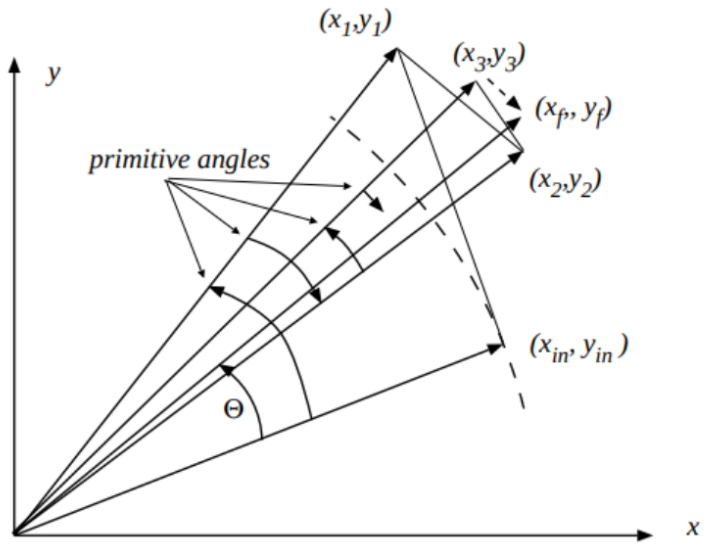


Figure 1: Vector rotation using microrotations.

2.2 Scaling Factor Compensation

The rotation-extension scales the modulus by $M[j+1] = \sqrt{1+2^{-2j}}M[j]$. The total scaling factor after ∞ iterations converges to a constant, independent of the target angle:

$$K = \prod_{j=0}^{\infty}(1+2^{-2j})^{1/2} \approx 1.6468 \tag{10}$$

To compute true rotations, this scaling factor must be compensated. Common methods include:

- **Direct Multiplication**: Multiply the final results by $1/K \approx 0.60725$.
- **Scaling Iterations**: Applying steps of the form $x_s = x \pm x \cdot 2^{-i}$.
- **Repeated Iterations**: Repeating certain CORDIC iterations to force the product to reach a known power of two or a stable target. For $n = 24$, repetitions include iterations 1, 3, 5, 6.

3 Unified Description

The CORDIC algorithm can be generalized to three coordinate systems using a parameter m:

- Circular ($m = 1$)
- Linear ($m = 0$)
- Hyperbolic ($m = -1$)

The unified microrotation is defined as:

$$x[j+1] = x[j] - m\sigma_j 2^{-j} y[j] \tag{11}$$

$$y[j+1] = y[j] + \sigma_j 2^{-j} x[j] \tag{12}$$

$$z[j+1] = \begin{cases} z[j] - \sigma_j \tan^{-1}(2^{-j}) & \text{if } m = 1 \\ z[j] - \sigma_j \tanh^{-1}(2^{-j}) & \text{if } m = -1 \\ z[j] - \sigma_j(2^{-j}) & \text{if } m = 0 \end{cases} \tag{13}$$

The specific initial values and rotation/vectoring mode outputs vary according to the coordinate system. Note that hyperbolic CORDIC ($m = -1$) must begin at $j = 1$ (since $\tanh^{-1}(2^0) = \infty$) and requires specific iterations (e.g., 4, 13, 40) to be repeated to guarantee convergence.

4 Operating Modes (Circular, $m = 1$)

4.1 Rotation Mode

Used to rotate a vector by a specific angle θ. We start with $z[0] = \theta$.

- **Objective**: Reduce z to 0.
- **Decision**: $\sigma_j = \text{sign}(z[j])$.
- **Result**: $x_f = K(x_{in}\cos\theta - y_{in}\sin\theta)$, $y_f = K(x_{in}\sin\theta + y_{in}\cos\theta)$, $z_f = 0$.

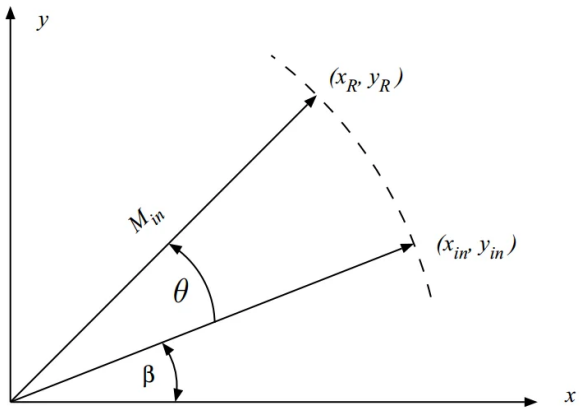


Figure 2: CORDIC Rotation Mode: Iteratively rotating a vector towards a target angle.

4.2 Vectoring Mode

Used to find the magnitude and angle (phase) of a vector (x_{in}, y_{in}).

- **Objective**: Reduce y to 0.
- **Decision**: $\sigma_j = -\text{sign}(y[j])$.
- **Result**: $x_f = K\sqrt{x_{in}^2 + y_{in}^2}$, $y_f = 0$, $z_f = z_{in} + \tan^{-1}(y_{in}/x_{in})$.

5 Implementations

CORDIC implementations generally fall into two architectural categories:

Word-Serial Hardware consisting of a single set of adders/subtractors and variable barrel shifters. The hardware is reused sequentially across N clock cycles. This is resource-efficient but has lower throughput and requires routing multiplexers.

Pipelined An unrolled hardware implementation with N stages. Each stage corresponds to an iteration j with hardwired shifts instead of barrel shifters. Highly efficient for streaming data (1 result per clock cycle after initial latency) but area-intensive.

6 Redundant Representation and High Radix

The critical path in a standard CORDIC implementation is the Carry-Propagate Adder (CPA) used to update x, y, z. To reduce this delay, redundant representation adders (e.g., carry-save) can be used.

However, redundant additions make strict sign evaluation difficult. If we use a redundant representation for z (or y), calculating $\sigma_j = \text{sign}(z[j])$ precisely is impossible without a fast CPA. One solution is to allow $\sigma_j \in \{-1, 0, 1\}$ based on an estimate. If $\sigma_j = 0$, however, the scaling factor $(1 + 0 \cdot 2^{-2j})^{1/2} = 1$, which makes the final K variable depending on the input data.

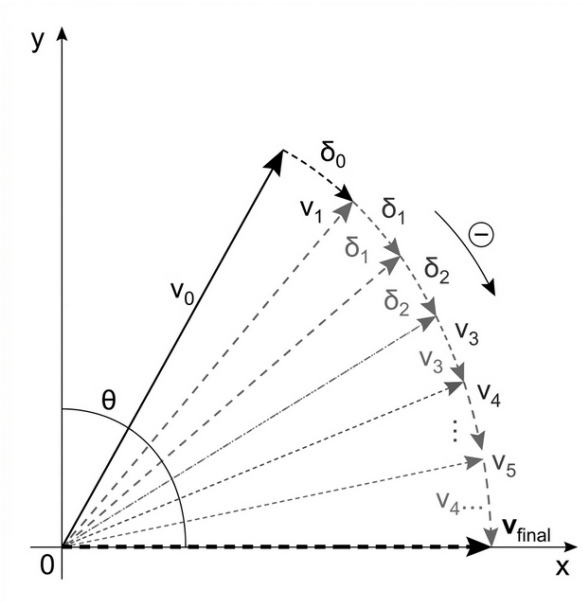


Figure 3: CORDIC Vectoring Mode: Iteratively rotating an arbitrary vector towards the x-axis.

6.1 Double Rotation Approach

To retain redundant arithmetic without variable scaling factors, the Double Rotation approach forces two identical rotations unconditionally. For each iteration step (j):

- If $\sigma_j = 1$, rotate by $\tan^{-1}(2^{-(j+1)})$ twice.
- If $\sigma_j = -1$, rotate by $-\tan^{-1}(2^{-(j+1)})$ twice.
- If $\sigma_j = 0$, rotate by $\tan^{-1}(2^{-(j+1)})$ and then $-\tan^{-1}(2^{-(j+1)})$.

This ensures that regardless of σ_j, the scaling magnitude applied during step j is strictly $1 + 2^{-2j-2}$, maintaining a constant K. The updated unified recurrence involves q_j and p_j terms determined by σ_j, requiring three-operand addition instead of two.